

# Java Backend Developer

8 Years Experience

**Last 1-Year Trending Interview Questions**

Spring Boot · Microservices · JVM Internals · System Design · Cloud & DevOps

Compiled for senior/lead engineers · Covers real questions from FAANG, startups & product companies (2024–2025)

## ■ Table of Contents

- 1. Java Core & JVM Internals
- 2. Concurrency & Multithreading
- 3. Spring Boot & Spring Ecosystem
- 4. Microservices & Distributed Systems
- 5. Database & JPA / Hibernate
- 6. Messaging: Kafka & RabbitMQ
- 7. REST API Design & Security
- 8. System Design & Scalability
- 9. Cloud, Docker & Kubernetes
- 10. Coding & Streams (Java 8+)

## 1. Java Core & JVM Internals

Q1.

**Explain the JVM memory model: heap vs. non-heap. How does G1GC work differently from ZGC?**

Heap (Young Gen → Eden + S0/S1, Old Gen) holds object instances. Non-heap holds Metaspace (class metadata), code cache, thread stacks.

G1GC divides heap into equal-size regions; collects regions with most garbage first (hence 'Garbage-First'). Pause time is configurable.

ZGC is a concurrent, low-latency GC (sub-10 ms pauses) that uses colored pointers and load barriers. Best for large heaps (>16 GB).

✓ *Interviewer tip: Be ready to discuss -XX:+UseZGC vs -XX:+UseG1GC tuning flags.*

---

Q2.

**What is the difference between String.intern(), String Pool, and heap String objects?**

String literals are automatically placed in the String Pool (inside heap since Java 7). intern() moves a heap String into the pool or returns the existing reference.

Consequence: '==' works for interned strings; always use .equals() for safety.

```
String a = new String("hi"); // heap
String b = a.intern(); // pool → same ref as "hi" literal
```

---

Q3.

**How does the Java Memory Model (JMM) define 'happens-before' relationships?**

The JMM guarantees that if action A happens-before B, then A's results are visible to B.

Key rules: monitor lock/unlock, volatile write/read, Thread.start()/join(), static initialiser completion.

✓ *Interviewer tip: Common follow-up: why double-checked locking needs volatile.*

---

Q4.

**What changed in Java 17–21 that a senior engineer must know?**

Sealed classes (17), Pattern matching for switch (21 GA), Virtual Threads / Project Loom (21 GA), Record patterns, Sequenced Collections.

Virtual threads: lightweight threads scheduled by the JVM; eliminates the 'one-thread-per-request' bottleneck for IO-bound workloads.

✓ *Interviewer tip: Mention structured concurrency (preview in 21) if asked about Project Loom.*

---

## 2. Concurrency & Multithreading

**Q5.**

### **CompletableFuture vs. Virtual Threads — when would you choose each?**

CompletableFuture is ideal for composing async pipelines (thenApply, thenCompose, allOf). Good pre-Java21.

Virtual Threads (Java 21) let you write synchronous-looking blocking code without sacrificing throughput. Prefer for IO-heavy services.

```
// Virtual Thread executor (Java 21)
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {
    exec.submit(() -> callExternalService());
}
```

---

**Q6.**

### **Explain ConcurrentHashMap internals. How does it differ from Collections.synchronizedMap()?**

ConcurrentHashMap uses segment-level locking (Java 7) / CAS + synchronized on bin head (Java 8+). Reads are lock-free.

synchronizedMap wraps every method with a single mutex — far less concurrency. Iterators on synchronizedMap require external sync.

✓ *Interviewer tip: Ask about compute(), merge(), forEach() on CHM for bonus points.*

---

**Q7.**

### **What is a StampedLock and when is it better than ReadWriteLock?**

StampedLock supports optimistic reads: tryOptimisticRead() returns a stamp; validate(stamp) checks if a write occurred.

If valid, no lock was held → extremely fast for read-heavy, write-rare scenarios. Falls back to a full read lock if invalid.

---

**Q8.**

### **How do you detect and fix a deadlock in a production Java service?**

Detection: thread dump (jstack / kill -3), look for 'BLOCKED' threads with circular lock dependencies.

Fix strategies: consistent lock ordering, tryLock() with timeout, use higher-level concurrency primitives (Semaphore, ReentrantLock).

✓ *Interviewer tip: Mention async.profiler or Java Flight Recorder for production diagnosis.*

---

### 3. Spring Boot & Spring Ecosystem

**Q9.**

**Explain Spring Boot auto-configuration internals. How does @ConditionalOnMissingBean work?**

Auto-config classes are listed in

META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports.

@ConditionalOnMissingBean registers a bean only if no bean of that type exists in the context — enables user overrides without touching library code.

---

**Q10.**

**How would you implement distributed transaction management across microservices?**

SAGA pattern: sequence of local transactions with compensating actions on failure.

Two flavours: Choreography (events) and Orchestration (central saga orchestrator).

Two-phase commit (2PC) is possible with JTA but couples services — avoid in microservices.

✓ *Interviewer tip: Mention Axon Framework or Spring Modulith for saga orchestration.*

---

**Q11.**

**What is the difference between @Transactional propagation REQUIRED vs REQUIRES\_NEW?**

REQUIRED (default): joins existing transaction or starts one. REQUIRES\_NEW: always suspends current tx and starts fresh.

Use REQUIRES\_NEW for audit logging — you want the log persisted even if the outer tx rolls back.

---

**Q12.**

**How do you secure a Spring Boot API with OAuth2 + JWT? Walk through the filter chain.**

Resource Server validates JWT signature using public key / JWKS endpoint of Auth Server.

Filters: JwtAuthenticationFilter extracts bearer token → JwtDecoder validates → SecurityContextHolder populated.

Key annotations: @EnableWebSecurity, @EnableMethodSecurity, @PreAuthorize("hasRole('ADMIN')").

---

## 4. Microservices & Distributed Systems

**Q13.**

**How do you implement the Circuit Breaker pattern? Compare Resilience4j vs Hystrix.**

Circuit Breaker: CLOSED (normal) → OPEN (failures exceed threshold, fast-fail) → HALF-OPEN (probe).

Hystrix is in maintenance; Resilience4j is lightweight, modular, and designed for Java 8+ functional APIs.

Resilience4j also provides: RateLimiter, Retry, Bulkhead, TimeLimiter.

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallback")
public Order placeOrder(OrderRequest req) { ... }
```

---

**Q14.**

**What is eventual consistency and how do you handle it in a microservices architecture?**

Services communicate asynchronously via events. Each service updates its own DB; data converges over time.

Challenges: out-of-order events, duplicate events (idempotency key), read-your-writes.

Patterns: outbox pattern (write event to same DB as business data), CDC with Debezium.

---

**Q15.**

**Explain API Gateway vs Service Mesh (Istio). When do you need both?**

API Gateway (Kong, AWS API GW): north-south traffic — auth, rate limiting, routing at the edge.

Service Mesh (Istio, Linkerd): east-west traffic — mTLS between services, observability, retries at the sidecar level.

Use both when you need edge-level auth AND fine-grained inter-service policies.

---

## 5. Database & JPA / Hibernate

**Q16.**

**What is the N+1 query problem in Hibernate? How do you fix it?**

N+1: fetching a list of N entities then issuing N additional queries for each association.

Fixes: JOIN FETCH in JPQL, @EntityGraph, Hibernate batch fetching (hibernate.default\_batch\_fetch\_size).

```
@Query("SELECT o FROM Order o JOIN FETCH o.items WHERE o.user = :user")
List findByUser(@Param("user") User user);
```

✓ *Interviewer tip: Also mention Blaze-Persistence or projections/DTOs to avoid entity graph overhead.*

---

**Q17.**

**Explain database indexing strategies for a write-heavy vs read-heavy workload.**

Read-heavy: composite indexes, covering indexes, partial indexes. Use EXPLAIN ANALYZE.

Write-heavy: fewer indexes (each index slows INSERT/UPDATE), use write-optimised engines, partition tables, batch writes.

B-Tree for equality/range; Hash for equality only; GIN/GiST for JSON, full-text (PostgreSQL).

---

**Q18.**

**How do you handle optimistic vs pessimistic locking in JPA?**

Optimistic: @Version field — Hibernate checks version on UPDATE; throws OptimisticLockException on conflict. Best for low contention.

Pessimistic: LockModeType.PESSIMISTIC\_WRITE issues SELECT ... FOR UPDATE. Best for high contention / financial data.

---

## 6. Messaging: Kafka & RabbitMQ

**Q19.**

**Explain Kafka's exactly-once semantics (EOS). How is it achieved?**

EOS = idempotent producer + transactional API + read-committed isolation on consumer.

Idempotent producer: broker deduplicates using sequence numbers per producer epoch.

Transactions: `producer.beginTransaction()` / `commitTransaction()` — atomic write across partitions and consumer offsets.

✓ *Interviewer tip: Mention performance trade-off: EOS adds ~10–20% latency overhead.*

---

**Q20.**

**When would you choose RabbitMQ over Kafka?**

RabbitMQ: complex routing (topic/fanout/header exchanges), short-lived messages, RPC patterns, priority queues, lower ops overhead.

Kafka: high-throughput event streaming, event sourcing, replay capability, long retention, many consumers per topic.

---

**Q21.**

**How do you handle poison pill messages in Kafka consumers?**

Options: dead-letter topic (DLT), skip-and-log with `SeekToCurrentErrorHandler`, retry with exponential backoff (Spring Kafka `DefaultErrorHandler`).

Spring Kafka 2.8+: `DeadLetterPublishingRecoverer` sends failed records to .DLT automatically.

---

## 7. REST API Design & Security

**Q22.**

**What are OWASP Top 10 vulnerabilities a Java backend must defend against?**

Key ones for APIs: Broken Access Control, Injection (SQL/NoSQL/LDAP), Security Misconfiguration, Cryptographic Failures, SSRF.

Defences: input validation, parameterised queries, RBAC/ABAC, secrets in Vault (not code), HTTPS everywhere, CSP headers.

---

**Q23.**

**How do you design an idempotent REST API for payment processing?**

Client sends an Idempotency-Key header (UUID). Server stores key → response in a distributed cache (Redis) with TTL.

On duplicate request: return cached response without re-processing. Key must be scoped per user to prevent hijacking.

---

**Q24.**

**Explain the difference between authentication and authorisation. How does Spring Security handle both?**

Authentication: who are you? (JWT, Basic, OAuth2). Authorisation: what can you do? (roles, permissions).

Spring Security: AuthenticationManager → AuthenticationProvider (e.g., DaoAuthenticationProvider) → SecurityContext.

Authorisation: AccessDecisionManager / MethodSecurityInterceptor for @PreAuthorize expressions.

---



## 8. System Design & Scalability

**Q25.**

**Design a rate limiter for a public API handling 100k requests/sec.**

Algorithms: Token Bucket (bursty traffic friendly), Sliding Window Counter, Fixed Window.

Implementation: Redis + Lua script for atomic decrement; or use API Gateway built-in rate limiting.

Distribute counters per user/IP across Redis cluster; use local in-memory cache for hot keys to reduce Redis RTT.

✓ *Interviewer tip: Mention Bucket4j library for Java-level rate limiting.*

---

**Q26.**

**How do you design a notification service that must deliver 10M push notifications per day?**

Fan-out on write to Kafka topics partitioned by user segment.

Worker pool consumes, batches by device type (FCM/APNS), calls provider APIs with retry + exponential backoff.

Ensure idempotency; track delivery status in Cassandra for high write throughput.

---

**Q27.**

**Explain CAP theorem with a real-world example from your stack.**

CAP: in a network partition you choose Consistency or Availability — not both.

CP example: ZooKeeper, etcd (used in Kubernetes) — refuse writes on partition to stay consistent.

AP example: Cassandra, DynamoDB — return possibly stale data but stay available.

---

## 9. Cloud, Docker & Kubernetes

**Q28.**

### How do you optimise a Java Docker image for production?

Multi-stage build: compile in JDK image, run in minimal JRE (eclipse-temurin:21-jre-alpine).

Use jlink to create a custom minimal JRE. Enable CDS (Class Data Sharing) for faster startup.

Set -XX:MaxRAMPercentage=75 instead of fixed -Xmx to respect container limits.

```
FROM eclipse-temurin:21-jdk AS build
RUN ./mvnw package -DskipTests
FROM eclipse-temurin:21-jre-alpine
COPY --from=build /app/target/app.jar /app.jar
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app.jar"]
```

---

**Q29.**

### Explain Kubernetes liveness vs readiness vs startup probes for a Spring Boot app.

Liveness: is the app alive? Failing → pod restart. Use /actuator/health/liveness.

Readiness: is the app ready to serve? Failing → removed from Service endpoints. Use /actuator/health/readiness.

Startup: gives slow-starting apps extra time before liveness kicks in. Critical for apps with long init.

---

**Q30.**

### How do you manage secrets in a Kubernetes-deployed Java service?

Never store secrets in images or ConfigMaps. Use: Kubernetes Secrets (base64, not encrypted by default) → enable etcd encryption at rest.

Better: HashiCorp Vault with Vault Agent sidecar or Vault Secrets Operator. Or cloud-native: AWS Secrets Manager + External Secrets Operator.

---

## 10. Coding & Java Streams (Java 8+)

**Q31.**

**Find the top 3 most frequently occurring words in a large text file using Java Streams.**

Classic interview question combining streams, collectors, and sorting:

```
Files.lines(Path.of("file.txt"))
    .flatMap(line -> Arrays.stream(line.split("\\s+")))
    .map(String::toLowerCase)
    .collect(Collectors.groupingBy(w -> w, Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.comparingByValue().reversed())
    .limit(3)
    .forEach(System.out::println);
```

---

**Q32.**

**Implement a custom Collector that partitions a stream into chunks of size N.**

Streams don't natively support windowing — use a custom Collector:

```
public static Collector<T, List<T>, List<T>>
    chunked(int size) {
    return Collector.of(
        ArrayList::new,
        (acc, e) -> {
            if (acc.isEmpty() || acc.get(acc.size()-1).size() == size)
                acc.add(new ArrayList<>());
            acc.get(acc.size()-1).add(e);
        },
        (a, b) -> { a.addAll(b); return a; }
    );
}
```

✓ *Interviewer tip: Follow-up: how would you make this parallel-safe?*

---

**Q33.**

**Given a list of employees, find the highest-paid employee in each department.**

```
Map<String, Employee> result = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))
    ));
```

---

## 10b. More Coding Challenges (Java 8+)

**Q34.**

**Reverse words in a sentence without reversing individual characters using Streams.**

Split on spaces, collect into a list, reverse it, then join — all in one stream pipeline:

```
String input = "Hello World from Java";
String result = Arrays.stream(input.split(" "))
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        list -> { Collections.reverse(list); return list.stream(); }
    ))
    .collect(Collectors.joining(" "));
// Output: "Java from World Hello"
```

✓ *Interviewer tip: Follow-up: implement without Collections.reverse() using a Deque.*

---

**Q35.**

**Find all duplicate elements in a List using Streams.**

Use groupingBy + counting, then filter counts > 1:

```
List nums = List.of(1, 2, 3, 2, 4, 3, 5);

Set duplicates = nums.stream()
    .collect(Collectors.groupingBy(n -> n, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toSet());
// Output: [2, 3]
```

✓ *Interviewer tip: Alternative O(n) approach: add to HashSet, if add() returns false → duplicate.*

---

**Q36.**

**Flatten a List<List> and find the sum of all even numbers.**

flatMap() collapses nested lists; then filter + mapToInt for arithmetic:

```
List<List> nested = List.of(
    List.of(1, 2, 3), List.of(4, 5, 6), List.of(7, 8));

int sum = nested.stream()
    .flatMap(Collection::stream)
    .filter(n -> n % 2 == 0)
    .mapToInt(Integer::intValue)
    .sum();
// Output: 20 (2+4+6+8)
```

---

**Q37.**

**Implement a thread-safe Singleton using double-checked locking and explain why volatile is required.**

Without volatile, the JVM may partially construct the object before assigning the reference:

```
public class Singleton {
    private static volatile Singleton instance;

    private Singleton() {}

    public static Singleton getInstance() {
        if (instance == null) { // 1st check (no lock)
            synchronized (Singleton.class) {
                if (instance == null) { // 2nd check (with lock)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}

// Better alternative: use enum or static inner holder class
```

✓ Interviewer tip: Prefer Bill Pugh / enum Singleton in production — simpler and JVM-guaranteed safe.

---

**Q38.**

**Given a String, check if it is a palindrome ignoring spaces and case — using Streams.**

Stream over chars, filter letters, compare forward and reversed:

```
public static boolean isPalindrome(String s) {
    String cleaned = s.toLowerCase()
        .chars()
        .filter(Character::isLetterOrDigit)
        .collect(StringBuilder::new,
            StringBuilder::appendCodePoint,
            StringBuilder::append)
        .toString();
    return cleaned.equals(new StringBuilder(cleaned).reverse().toString());
}

// isPalindrome("A man a plan a canal Panama") → true
```

---

**Q39.****Find the first non-repeating character in a String using LinkedHashMap and Streams.**

LinkedHashMap preserves insertion order — combine with groupingBy counting:

```
public static char firstUnique(String s) {
    return s.chars()
        .mapToObj(c -> (char) c)
        .collect(Collectors.groupingBy(
            c -> c,
            LinkedHashMap::new, // preserve insertion order
            Collectors.counting()
        ))
        .entrySet().stream()
        .filter(e -> e.getValue() == 1)
        .map(Map.Entry::getKey)
        .findFirst()
        .orElseThrow(() -> new RuntimeException("None found"));
}

// firstUnique("aabbccde") → c
```

✓ Interviewer tip:  $O(n)$  time and  $O(1)$  space alternative: `int[26]` frequency array.

---

**Q40.****Group anagrams together from a List using Streams.**

Sort each word's chars as a key; group by that canonical key:

```
List words = List.of("eat", "tea", "tan", "ate", "nat", "bat");

Map> anagrams = words.stream()
    .collect(Collectors.groupingBy(w -> {
        char[] ch = w.toCharArray();
        Arrays.sort(ch);
        return new String(ch);
    }));

// {aet=[eat, tea, ate], ant=[tan, nat], abt=[bat]}
```

✓ Interviewer tip: Follow-up: use frequency `int[26]` as key for non-ASCII safe alternative.

---

#### Q41.

#### Implement a producer-consumer using BlockingQueue without explicit synchronization.

BlockingQueue handles all wait/notify internally — clean, thread-safe:

```
BlockingQueue queue = new LinkedBlockingQueue<>(10);

// Producer
Runnable producer = () -> {
    for (int i = 0; i < 20; i++) {
        try { queue.put(i); } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
    }
};

// Consumer
Runnable consumer = () -> {
    while (true) {
        try {
            Integer val = queue.poll(1, TimeUnit.SECONDS);
            if (val == null) break; // timeout = done
            System.out.println('Consumed: ' + val);
        } catch (InterruptedException e) { Thread.currentThread().interrupt(); break; }
    }
};

new Thread(producer).start();
new Thread(consumer).start();
```

✓ Interviewer tip: In Java 21, prefer virtual threads + SynchronousQueue for simpler handoff semantics.

---

#### Q42.

#### Compute Fibonacci numbers lazily using an infinite Stream.

Stream.iterate() with a seed pair generates an infinite sequence on demand:

```
Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
    .limit(15)
    .map(f -> f[0])
    .forEach(System.out::println);

// 0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

✓ Interviewer tip: Follow-up: memoize with a Map for recursive top-down DP approach.

---

**Q43.**

**Given a list of transactions, find the total amount per currency where amount > 1000 — sorted descending.**

Combines filter, groupingBy, summingDouble, and sorted entry stream:

```
// Transaction record: String currency, double amount
Map result = transactions.stream()
    .filter(t -> t.amount() > 1000)
    .collect(Collectors.groupingBy(
        Transaction::currency,
        Collectors.summingDouble(Transaction::amount)
    ))
    .entrySet().stream()
    .sorted(Map.Entry.comparingByValue().reversed())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (a, b) -> a,
        LinkedHashMap::new // preserve sorted order
    ));
```

---



**Q44.****Implement LRU Cache in Java without using LinkedHashMap.**

Use a HashMap + Doubly Linked List — O(1) get and put:

```
class LRUCache {
    private final int cap;
    private final Map map = new HashMap<>();
    private final Node head = new Node(0,0), tail = new Node(0,0);

    LRUCache(int cap) {
        this.cap = cap;
        head.next = tail; tail.prev = head;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        Node n = map.get(key); remove(n); insertFront(n); return n.val;
    }

    public void put(int key, int val) {
        if (map.containsKey(key)) remove(map.get(key));
        if (map.size() == cap) { map.remove(tail.prev.key); remove(tail.prev); }
        insertFront(new Node(key, val));
    }

    private void remove(Node n) {
        map.remove(n.key);
        n.prev.next = n.next; n.next.prev = n.prev;
    }

    private void insertFront(Node n) {
        map.put(n.key, n);
        n.next = head.next; n.prev = head;
        head.next.prev = n; head.next = n;
    }

    static class Node { int key, val; Node prev, next;
        Node(int k, int v) { key=k; val=v; } }
}
```

✓ Interviewer tip: Quick version: extend LinkedHashMap with removeEldestEntry() override.

---

**Q45.****Check if two strings are anagrams of each other — three approaches with complexity.**

Compare sorted char arrays, or frequency counts — all  $O(n)$ :

```
// Approach 1: Sort ( $O(n \log n)$ )
boolean isAnagram1(String a, String b) {
    char[] ca = a.toCharArray(); char[] cb = b.toCharArray();
    Arrays.sort(ca); Arrays.sort(cb);
    return Arrays.equals(ca, cb);
}

// Approach 2: Frequency array ( $O(n)$ ,  $O(1)$  space for ASCII)
boolean isAnagram2(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] freq = new int[26];
    for (char c : a.toCharArray()) freq[c - 'a']++;
    for (char c : b.toCharArray()) if (--freq[c - 'a'] < 0) return false;
    return true;
}

// Approach 3: Stream + groupingBy (most readable)
boolean isAnagram3(String a, String b) {
    return a.chars().boxed()
        .collect(Collectors.groupingBy(c->c, Collectors.counting()))
        .equals(b.chars().boxed()
            .collect(Collectors.groupingBy(c->c, Collectors.counting())));
}
```